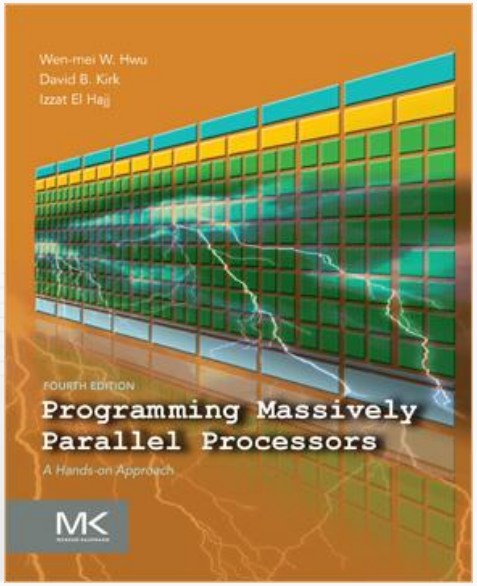
MORGAN KAUFMANN

# Programming Massively Parallel Processors

## A Hands-on Approach

**CHAPTER 2** › Heterogeneous Data Parallel Computing

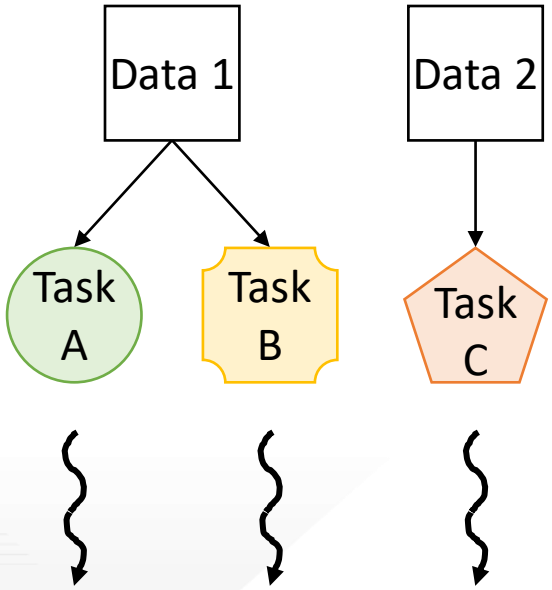



Copyright © 2022 Elsevier

# Types of Parallelism

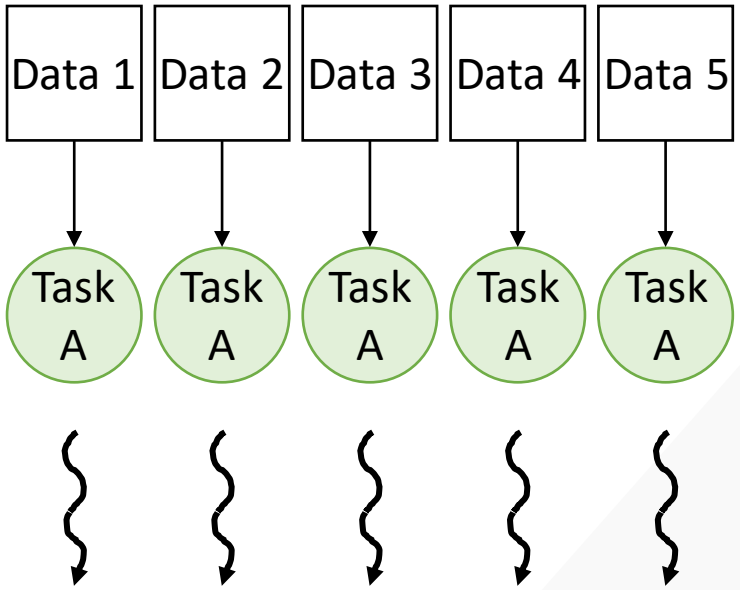
## Task Parallelism

- Different operations performed on same or different data
- Usually a modest number of tasks unleashing a modest amount of parallelism




## Data Parallelism

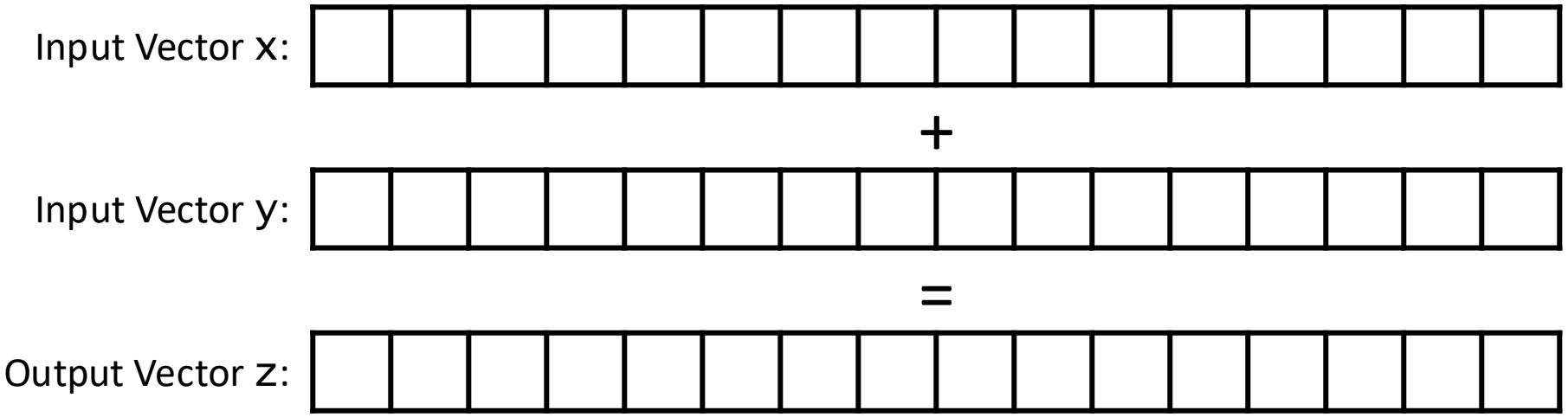
- Same operations performed on different data
- Potentially massive amounts of data unleashing massive amounts of parallelism
  - Most suitable for GPUs




Copyright © 2022 Elsevier

# Vector Addition

- Vector addition
  - The “Hello, World!” for data parallel programming

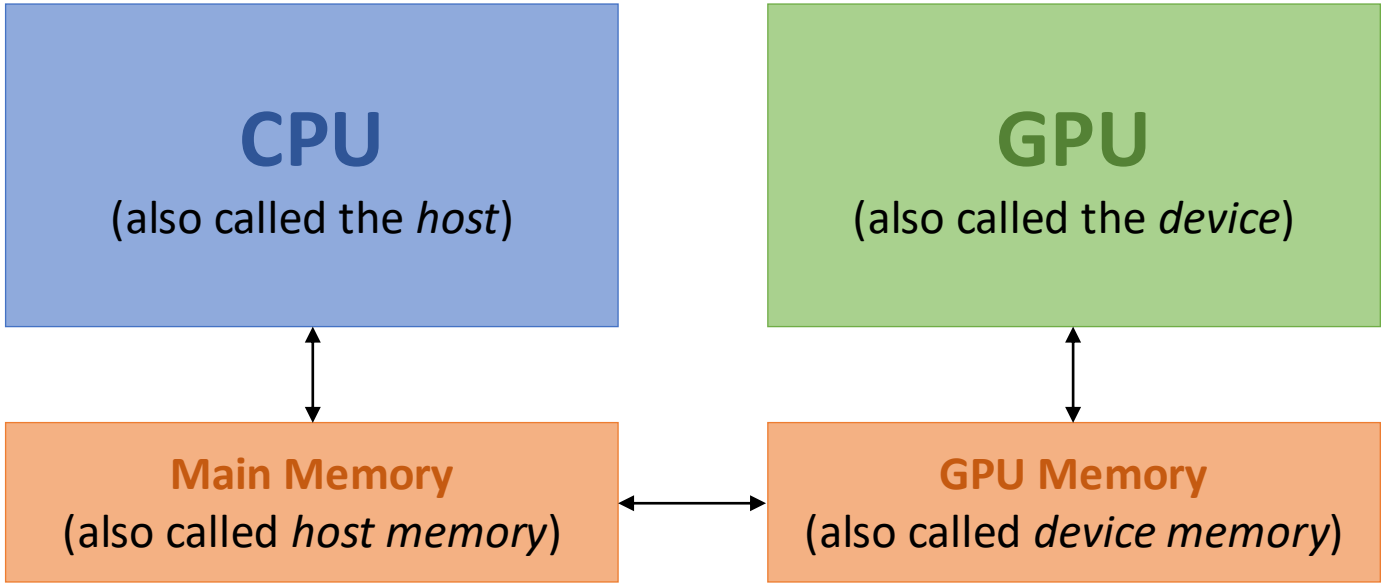



```
void vecadd(float* x, float* y, float* z, int N) {
    for(unsigned int i = 0; i < N; ++i) {
        z[i] = x[i] + y[i];
    }
}
```

Sequential C Code

Copyright © 2022 Elsevier

# System Organization

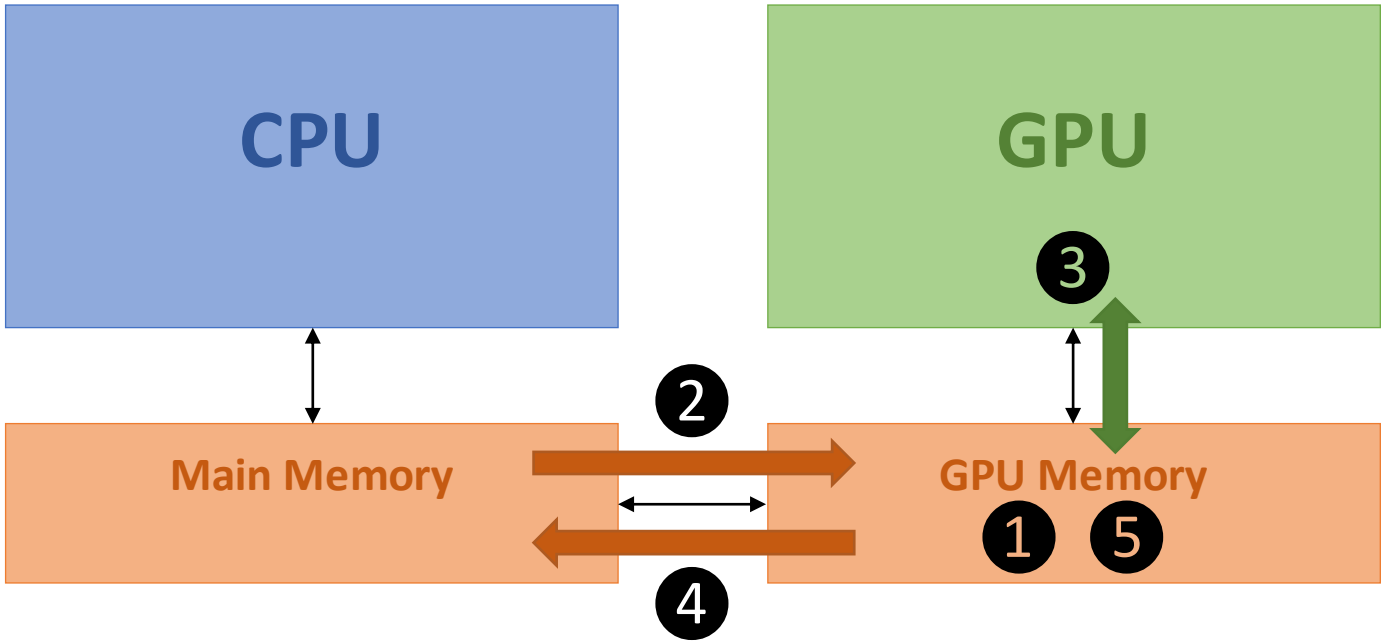



The CPU and GPU have separate memories
can cannot access each others' memories

Need to transfer data between them

Copyright © 2022 Elsevier

# Memory Management




1. Allocate GPU memory
2. Copy data to GPU memory
3. Perform computation on GPU
4. Copy data from GPU memory
5. Deallocate GPU memory

Copyright © 2022 Elsevier

# CUDA Memory Management API

- Allocating memory:

```
cudaError_t cudaMalloc(void **devPtr, size_t size)
```

  - `devPtr`: Pointer to pointer to allocated device memory
  - `size`: Requested allocation size in bytes

- Deallocating memory:

```
cudaError_t cudaFree(void *devPtr)
```

  - `devPtr`: Pointer to device memory to free

- Return type: `cudaError_t`
  - Helps with error checking (discussed later)

Copyright © 2022 Elsevier

# CUDA Memory Management API

- Copying memory:

```
cudaError_t cudaMemcpy(void *dst, const void *src,
                       size_t count, enum cudaMemcpyKind kind)
```

  - `dst`: Destination memory address
  - `src`: Source memory address
  - `count`: Size in bytes to copy
  - `kind`: Type of transfer
    - `cudaMemcpyHostToHost`
    - `cudaMemcpyHostToDevice`
    - `cudaMemcpyDeviceToHost`
    - `cudaMemcpyDeviceToDevice`

Copyright © 2022 Elsevier

# Code Example

```
void vecadd(float* x, float* y, float* z, int N) {

    // Allocate GPU memory
    float *x_d, *y_d, *z_d;
    cudaMalloc((void**) &x_d, N*sizeof(float));
    cudaMalloc((void**) &y_d, N*sizeof(float));
    cudaMalloc((void**) &z_d, N*sizeof(float));

    // Copy data to GPU memory
    cudaMemcpy(x_d, x, N*sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(y_d, y, N*sizeof(float), cudaMemcpyHostToDevice);

    // Perform computation on GPU
    ...

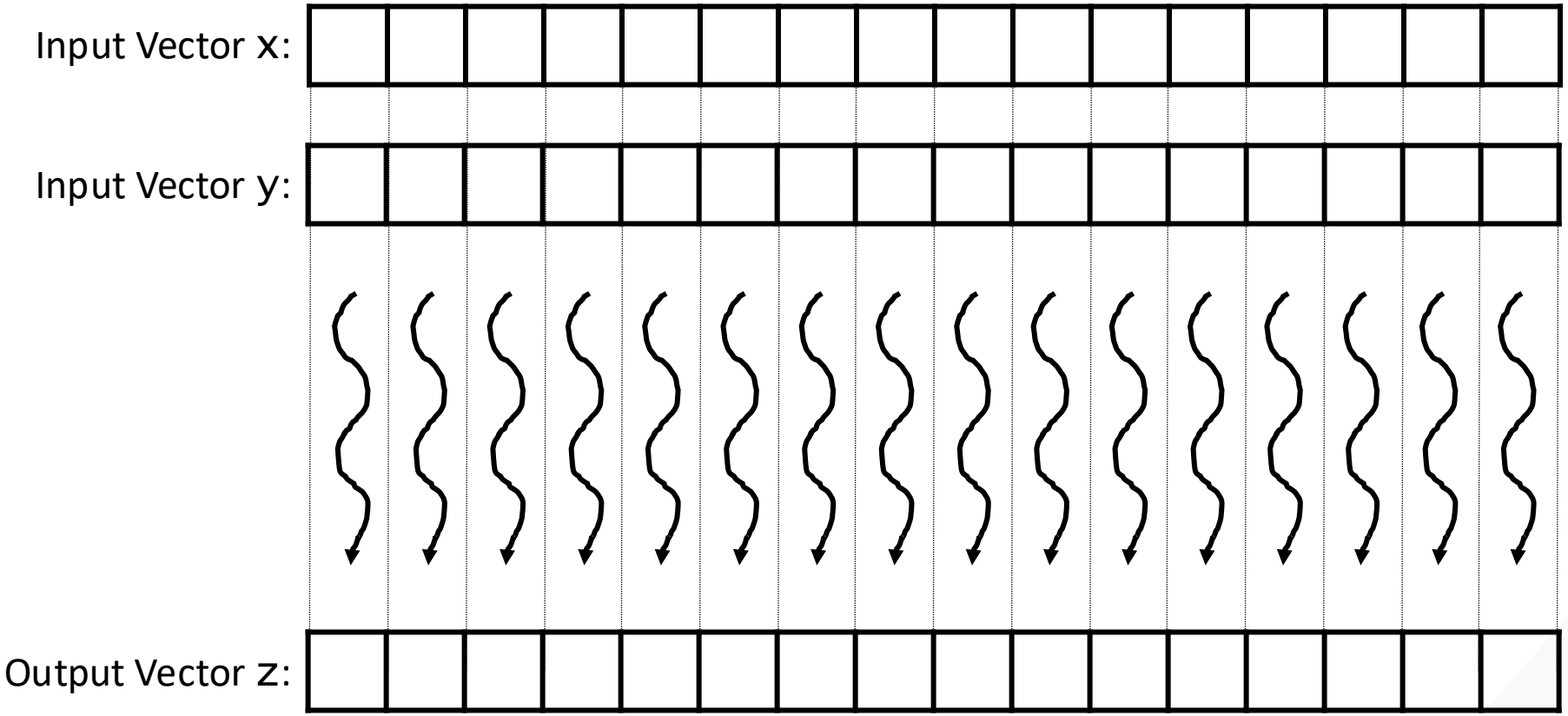
    // Copy data from GPU memory
    cudaMemcpy(z, z_d, N*sizeof(float), cudaMemcpyDeviceToHost);

    // Deallocate GPU memory
    cudaFree(x_d);
    cudaFree(y_d);
    cudaFree(z_d);

}
```

Copyright © 2022 Elsevier

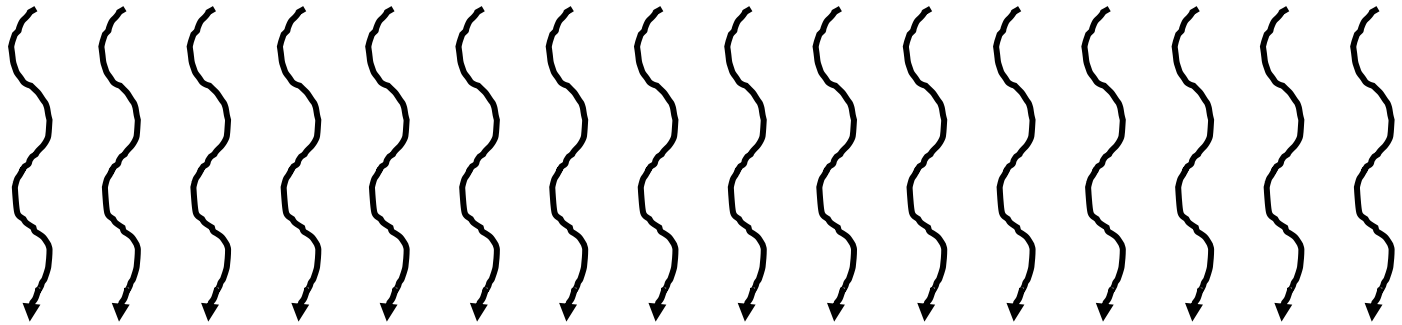
# Parallel Vector Addition




Simple strategy: assign one GPU **thread** per vector element

Copyright © 2022 Elsevier

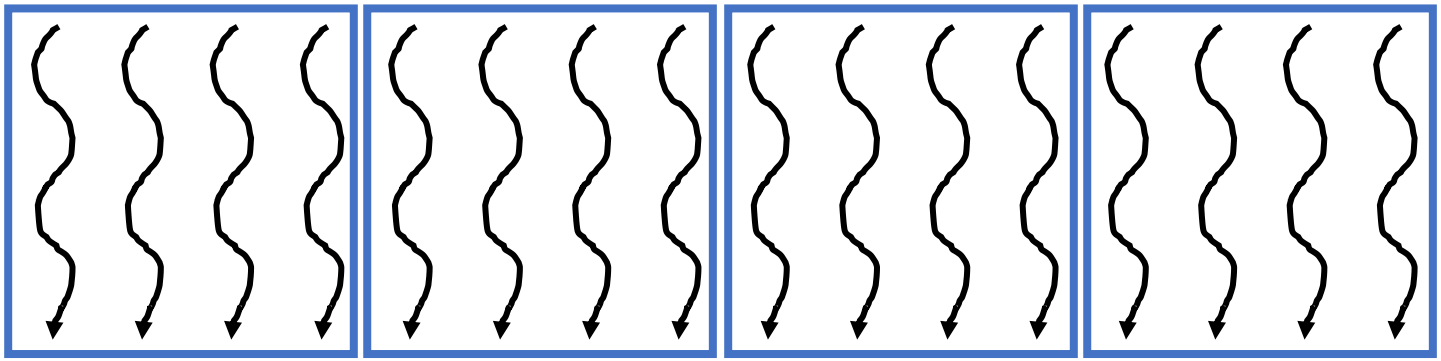
# Grids



An array of GPU threads is called a **grid**

Copyright © 2022 Elsevier

# Blocks



Threads in a grid are grouped into thread **blocks**

(significance: threads in the same block can collaborate in ways that threads in different blocks cannot – discussed later)

Copyright © 2022 Elsevier

# Launching a Grid

- Threads in the same grid execute the same function known as a **kernel**

- A grid can be launched by calling a kernel and configuring it with appropriate grid and block sizes

  - Example:

```
const unsigned int numThreadsPerBlock = 512;
const unsigned int numBlocks = N/numThreadsPerBlock;
vecadd_kernel <<< numBlocks, numThreadsPerBlock >>> (x_d, y_d, z_d, N);
```

Copyright © 2022 Elsevier

# Implementing a Kernel

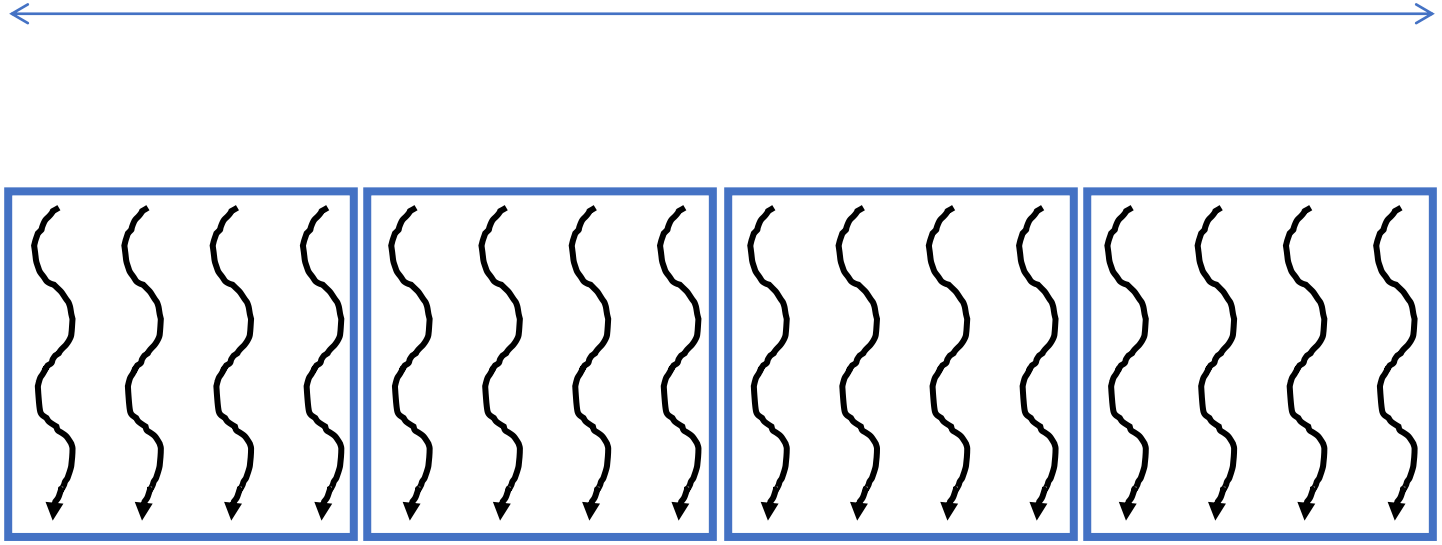
- A kernel is similar to a C/C++ function
- It is preceded by the keyword `__global__` to indicate that it is a GPU kernel
- It uses special keywords to distinguish different threads from each other

Copyright © 2022 Elsevier

# Grid Dimension

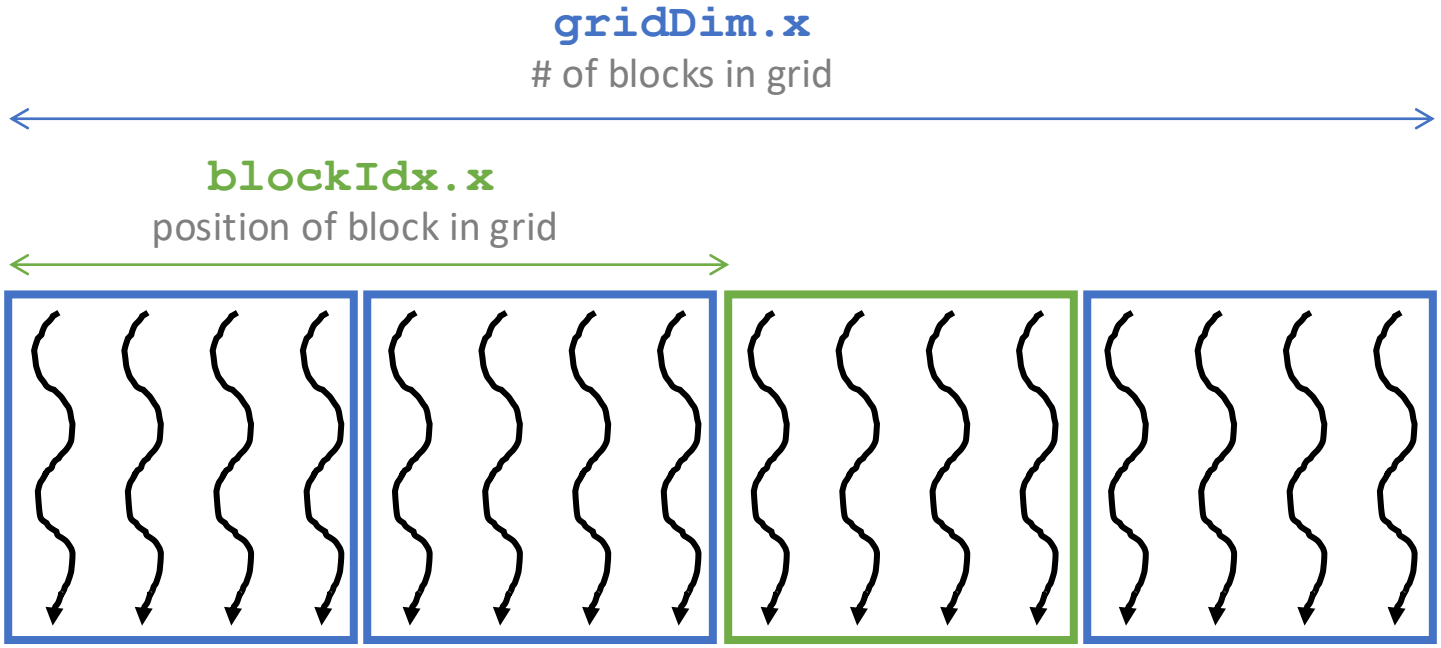
**`gridDim.x`**

# of blocks in grid



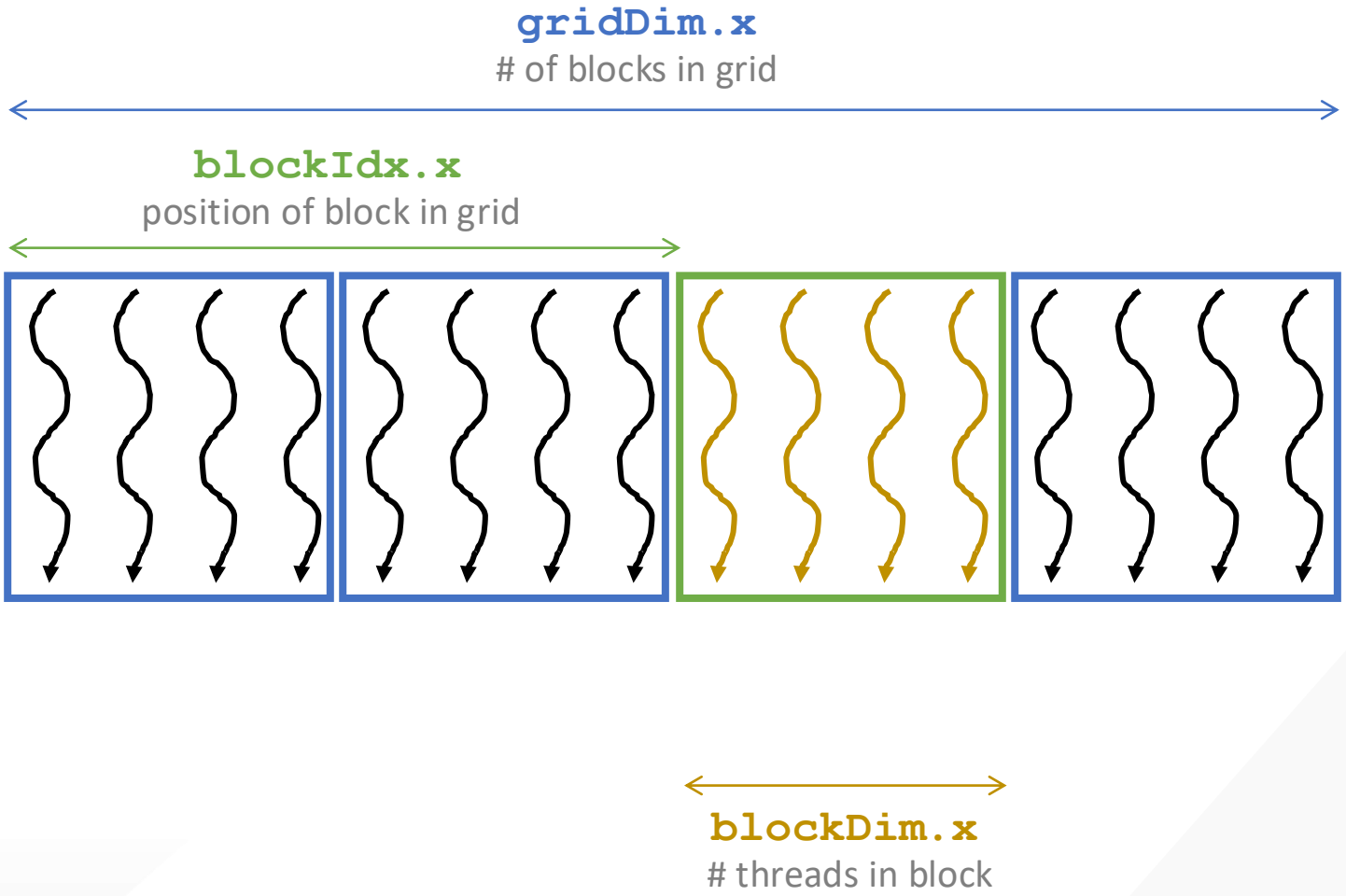
Copyright © 2022 Elsevier

# Block Index



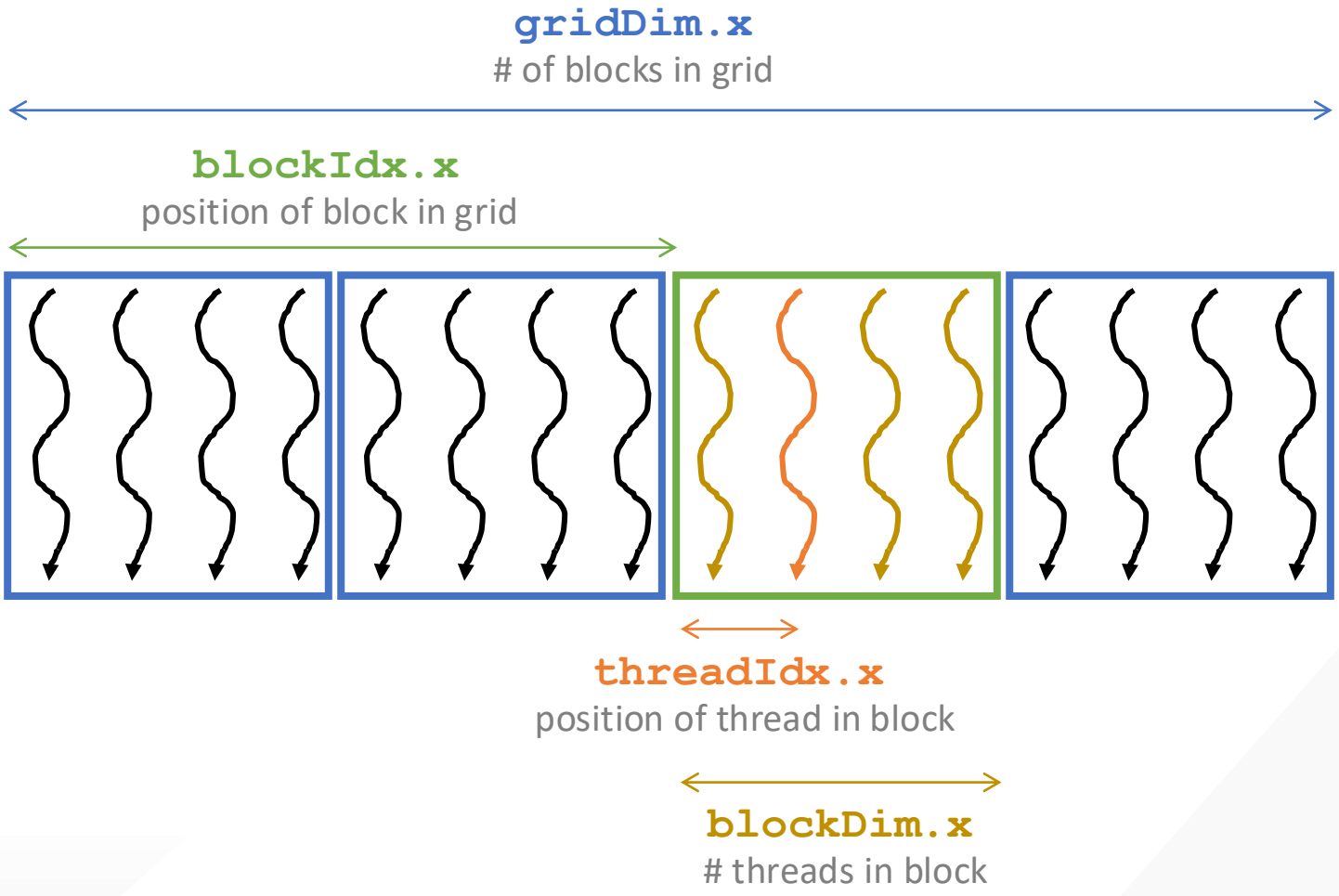

Copyright © 2022 Elsevier

# Block Dimension



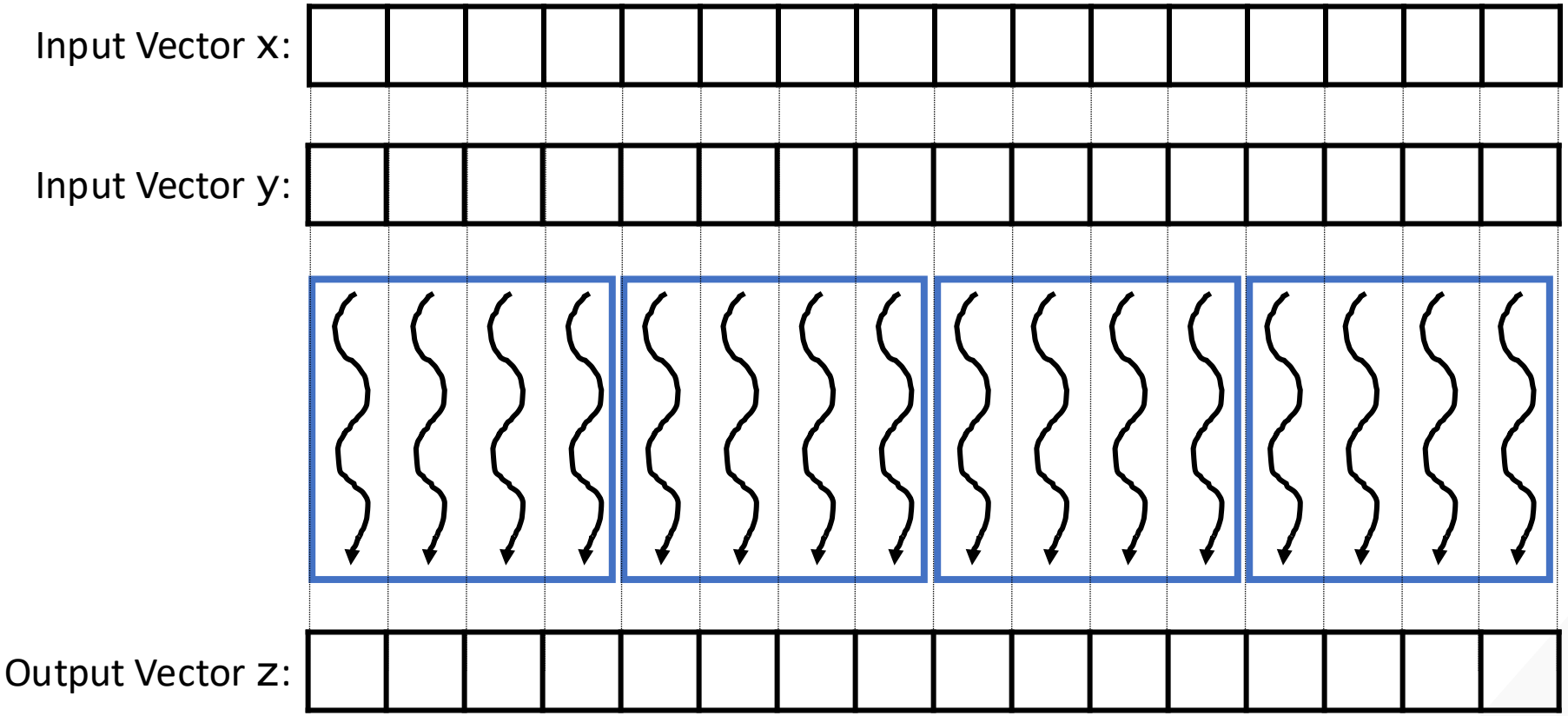

Copyright © 2022 Elsevier

# Thread Index




Copyright © 2022 Elsevier

# Parallel Vector Addition in CUDA




```
__global__ void vecadd_kernel(float* x, float* y, float* z, int N) {
    int i = blockDim.x*blockIdx.x + threadIdx.x;
    z[i] = x[i] + y[i];
}
```

Identify a thread's global index and use it to index the arrays

Copyright © 2022 Elsevier

# Boundary Conditions

- Recall launch configurations:

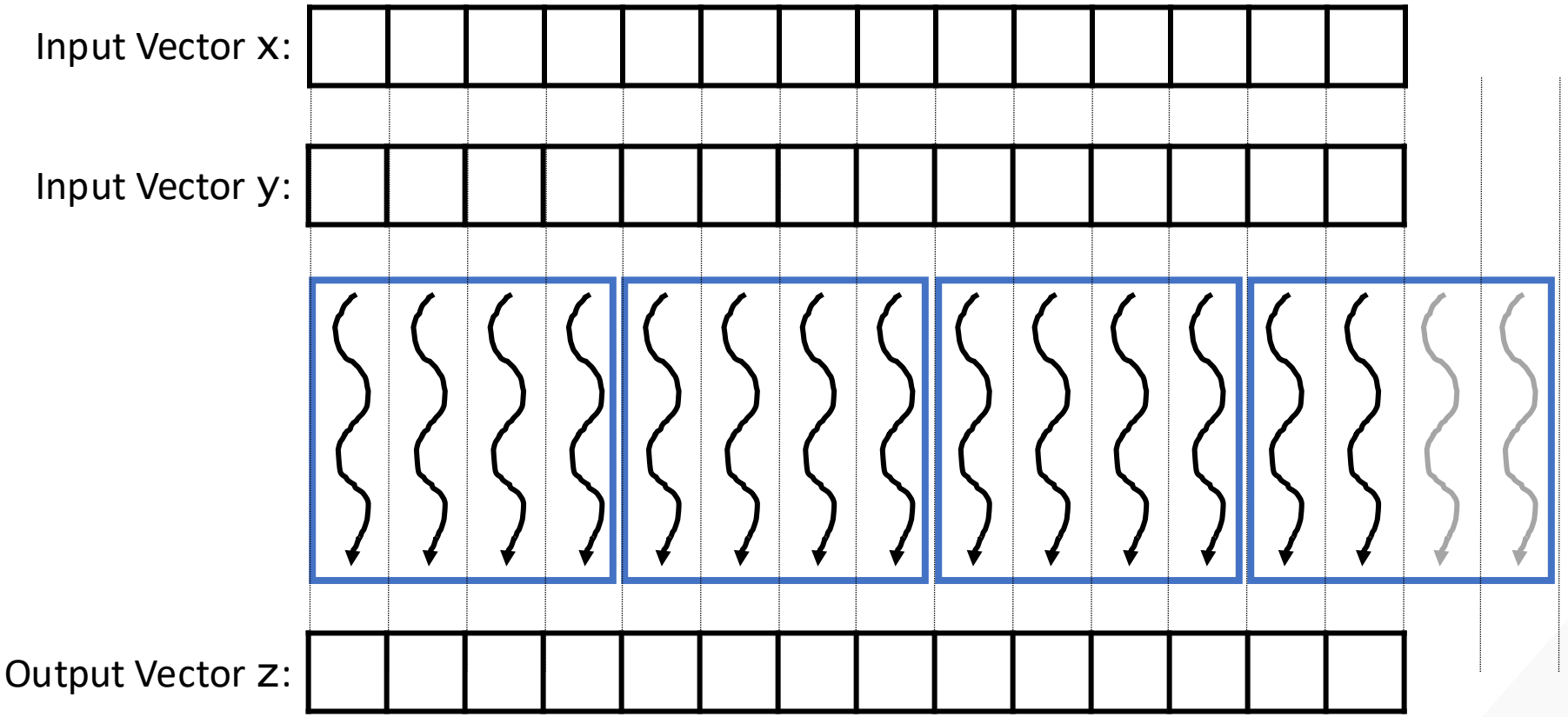
```
const unsigned int numThreadsPerBlock = 512;
const unsigned int numBlocks = N/numThreadsPerBlock;
vecadd_kernel <<< numBlocks, numThreadsPerBlock >>> (x_d, y_d, z_d, N);
```

- If N is not a multiple of `numThreadsPerBlock`, fewer threads will be launched than desired

- Solution: use the ceiling to launch extra threads then omit the threads after the boundary

Copyright © 2022 Elsevier

# Execution with Boundary Checks




Copyright © 2022 Elsevier

# Code with Boundary Checks

- Launch code:

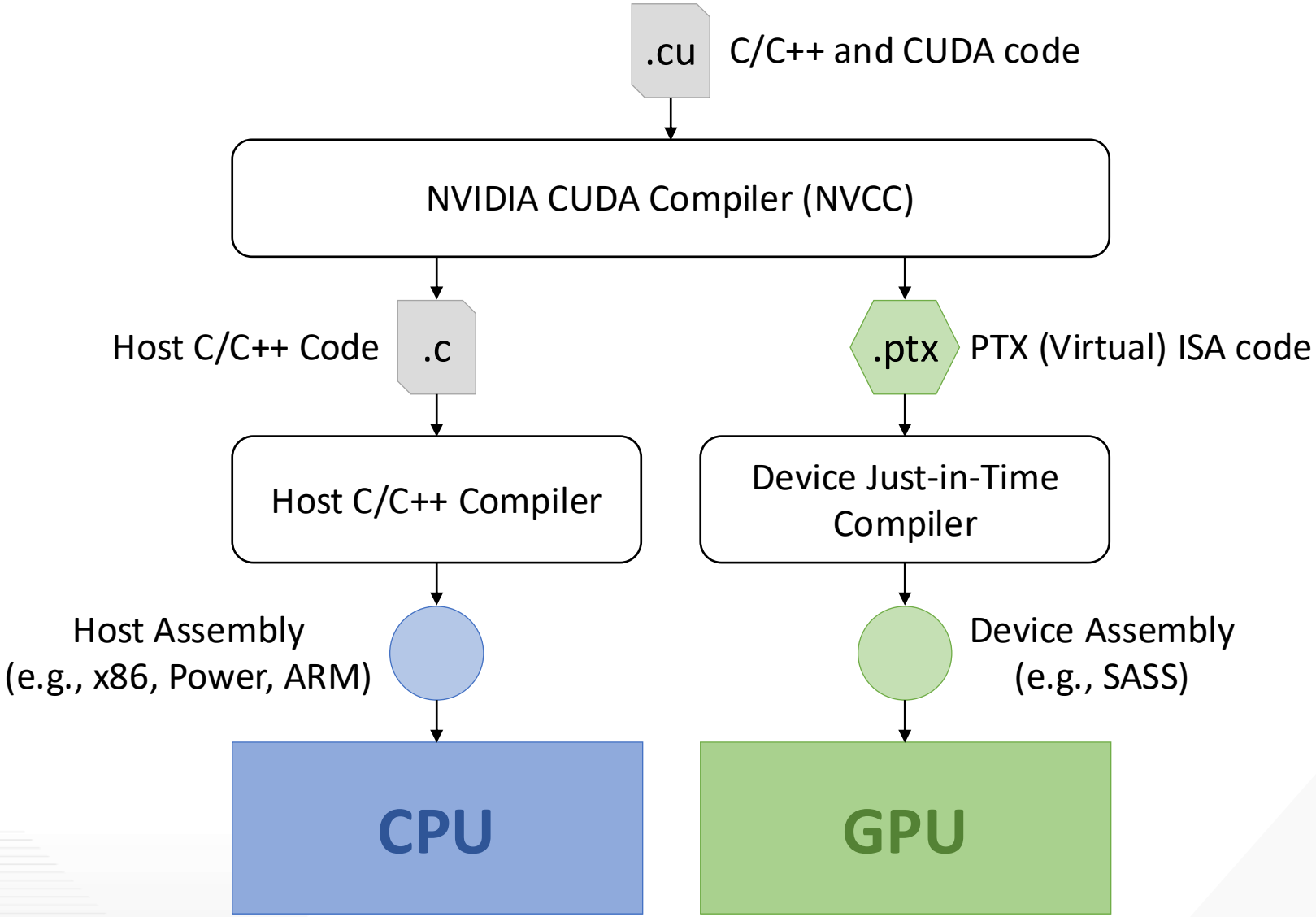
```
const unsigned int numBlocks = (N + numThreadsPerBlock - 1)/numThreadsPerBlock;
```

- Kernel code

```
__global__ void vecadd_kernel(float* x, float* y, float* z, int N) {
    int i = blockDim.x*blockIdx.x + threadIdx.x;
    if(i < N) {
        z[i] = x[i] + y[i];
    }
}
```

Copyright © 2022 Elsevier

# Compilation




Copyright © 2022 Elsevier

# Function Declarations

- Keywords that differentiate where a function is intended to run

| Keyword | Callable From | Executed On |
|---|---|---|
| __host__ (default) | Host | Host |
| __global__ | Host (or Device) | Device |
| __device__ | Device | Device |

- Why need `__host__` if it is the default?

Copyright © 2022 Elsevier

# Function Declarations

- The keyword `__host__` is useful when needing to mark a function as executable on both the host and the device

```
__host__ __device__ float f(float a, float b) {
    return a + b;
}
void vecadd(float* x, float* y, float* z, int N) {
    for(unsigned int i = 0; i < N; ++i) {
        z[i] = f(x[i], y[i]);
    }
}
__global__ void vecadd_kernel(float* x, float* y, float* z, int N) {
    int i = blockDim.x*blockIdx.x + threadIdx.x;
    if (i < N) {
        z[i] = f(x[i], y[i]);
    }
}
```

Copyright © 2022 Elsevier

# How much faster?

Copyright © 2022 Elsevier

# Asynchronous Kernel Calls

- By default, kernel calls are asynchronous
  - Useful for overlapping GPU computations with CPU computations

- Use the following API function to synchronize

  `cudaError_t cudaDeviceSynchronize()`
  - Blocks until the device has completed all preceding requested tasks

Copyright © 2022 Elsevier

# Error Checking

- All CUDA API calls return an error code `cudaError_t` that can be used to check if any errors occurred
  - Example:

```
cudaError_t err = ...;
if(err != cudaSuccess) {
    printf("Error: %s\n", cudaGetErrorString(err));
    exit(0);
}
```

- For kernel calls, one can check the error returned by `cudaDeviceSynchronize()` or call the following API function:

```
cudaError_t cudaGetLastError()
```

Copyright © 2022 Elsevier

# You just learned CUDA!

- The rest of this course is about parallelization strategies and optimizations for massively parallel processors

Copyright © 2022 Elsevier

# References


- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.

- CUDA C++ Programming Guide: https://docs.nvidia.com/cuda/cuda-c-programming-guide


Copyright © 2022 Elsevier